ID5130: Parallel Scientific Computing
Indian Institute of Technology Madras
January–May 2026

PARALLEL RAY-TRACING FOR ANALYTICALLY DEFINED OBJECTS WITH POINT CLASSIFICATION

Shubham Jadhav
CS24S009
Department of Computer Science and Engineering
Indian Institute of Technology Madras

ABSTRACT

Ray-tracing is image-order, embarrassingly parallel at the pixel level, and the workhorse of physically based rendering, yet a textbook implementation buckles under the cost of multi-bounce light transport. We present a from-scratch C++17 renderer for analytically defined primitives—sphere, plane, axis-aligned box, finite cylinder—and their Constructive Solid Geometry (CSG) combinations, with hybrid OpenMP and MPI parallelism on a CPU cluster. The renderer hosts two integrators on a shared geometric core: a Whitted recursive shader for hard-surface validation and a Monte-Carlo path tracer with Next-Event Estimation and Multiple Importance Sampling for global illumination. On the IIT Madras AQUA cluster (Intel Xeon Gold 6248, 4 nodes $\times$ 40 cores), OpenMP strong scaling on the Cornell Box at 1024×1024 *with* 256 *samples per pixel reaches a* $38.96\times$ *speed-up at* 40 *threads (97.4% parallel efficiency); the cross-node hybrid configuration with one MPI rank per socket attains* $65\times$ *on* 80 *physical cores. Output is byte-identical across thread and rank counts. Cornell Box rendering reproduces the colour-bleed signature qualitatively against a Mitsuba 3 reference.*

NOMENCLATURE

BVH	Bounding Volume Hierarchy.
CSG	Constructive Solid Geometry.
E_p	Parallel efficiency, S_p/p.
LPT	Longest-Processing-Time-first scheduling.
MIS	Multiple Importance Sampling.
NEE	Next-Event Estimation.
NUMA	Non-Uniform Memory Access.
PCG32	Permuted Congruential Generator, 32-bit output.
SAH	Surface Area Heuristic.
S_p	Speed-up at p workers, T_1/T_p.
SPP	Samples Per Pixel.
$\mathbf{r}(t)$	Parametric ray $\mathbf{o}+t\mathbf{d}$, $t \geq 0$.

INTRODUCTION

Motivation

Every image rendered for film, scientific visualisation or product design is an array of pixel colours synthesised from a mathematical description of a virtual scene. *Ray-tracing* produces such images by simulating the geometry of light transport: a ray is cast from the camera through each pixel, the closest intersection with the scene is identified, and a shading model converts that hit into a colour. Because the algorithm is geometric rather than approximated, ray-tracing handles shadows, mirror reflections, refractive transparency and diffuse interreflection as instances of the same primitive operation—recursive ray traversal—instead of the dedicated image-space heuristics required by rasterisation pipelines [1, 2]. The fidelity comes at a cost: a single Cornell-Box image of 10^6 pixels at 10^3 samples per pixel demands of order 10^9 ray-traversal events, each of which performs intersection mathematics and acceleration-structure descent. Parallel computing is therefore not optional but constitutive.

Problem Statement

Develop a parallel ray-tracing algorithm to display analytically defined objects on screen, and classify a given 3-D point as inside, on, or outside the object. Both halves reduce to evaluating an implicit function: along a ray for rendering, at a single point for classification. The Boolean algebra of CSG combines intervals along the ray or per-operand classifications under the same rules, so one set of primitive routines serves both queries.

Objectives

1. Implement a serial CPU ray-tracer for analytical primitives (sphere, plane, axis-aligned box, finite cylinder) and CSG operations (union, intersection, difference) using ray-interval merging.
2. Implement point-in-solid classification with explicit *inside / on / outside* states, including a tolerance-aware boundary case, for primitives, lists of primitives, and CSG nodes.
3. Embed the kernel inside two integrators that share scene, geometry and parallel infrastructure: a Whitted recursive shader and a Monte-Carlo path tracer with NEE+MIS for global illumination.
4. Parallelise the renderer with hybrid OpenMP (intra-node) and MPI (inter-node) and quantify performance with standard parallel metrics.
5. Validate visual correctness on the Cornell Box benchmark and quantify image quality against the Mitsuba 3 reference renderer using a high-sample-count ground truth.

BACKGROUND AND RELATED WORK

Image-Order vs Object-Order Rendering

Two paradigms dominate image synthesis. *Rasterisation* iterates over scene primitives and fills the pixels they cover; modern GPUs execute the rasterisation pipeline in dedicated hardware and dominate real-time graphics. *Ray-tracing* iterates over pixels and asks "what does the camera see along this direction?"—a query naturally decomposed into independent per-pixel work. The image-order structure is responsible for ray-tracing's combinatorial cost and, simultaneously, for its embarrassing parallelism: pixels read shared scene data and write disjoint pixel slots, requiring no synchronisation during the shading phase.

The Rendering Equation

Kajiya [2] formalised image synthesis as the integral equation

$$L_o(x,\omega_o) = L_e(x,\omega_o) + \int_\Omega f_r(x,\omega_i,\omega_o)\, L_i(x,\omega_i)\, (\omega_i \cdot \mathbf{n})\, d\omega_i. \quad (1)$$

The unknown radiance L appears on both sides, so closed-form solutions exist only for trivial scenes. Monte-Carlo path tracing replaces the hemispherical integral with stochastic sampling: a direction is drawn from a probability density, the recursion descends, and many such sample paths are averaged at the pixel. The variance of the estimator decays as $O(1/\sqrt{N})$ in the sample count N, which is the source of the SPP-dependent rendering cost.

Embarrassingly Parallel Workload

For a workload whose parallel fraction $p \to 1$, Amdahl's law [3]

$$S(P) = \frac{1}{(1-p) + p/P} \quad (2)$$

predicts almost linear speed-up while Gustafson's rescaling [4]

$$S(P) = P - (1-p)(P-1) \quad (3)$$

predicts effectively linear speed-up when the problem grows with P. Ray-tracing is the textbook example: scene setup and image output occupy a tiny fraction of the runtime, so realistic measured efficiency is bounded chiefly by load imbalance, NUMA placement and inter-node communication rather than by Amdahl's serial term.

Existing CPU Renderers

The dominant CPU renderers—PBRT-v4 [5], Mitsuba 3 (Dr.Jit) [6], and OSPRay/Embree [7]—all operate exclusively on triangle meshes; PBRT-v4 has no MPI back-end and Mitsuba 3 has no CSG. The combination of analytical primitives, CSG, and MPI execution is a niche that none of them occupies natively. Mitsuba 3 is used in this work as the cross-comparison reference.

GEOMETRIC KERNEL

Ray Model

A ray is the half-line $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ with $t \geq 0$. The renderer caches the component-wise reciprocal $\mathbf{d}^{-1}$ to avoid repeated divisions during AABB and slab tests.

Analytical Intersections

Sphere. A sphere with centre $\mathbf{c}$ and radius R satisfies $\|\mathbf{p} - \mathbf{c}\|^2 = R^2$. Substituting the ray and writing the resulting quadratic in normalised form yields

$$a = \mathbf{d}\cdot\mathbf{d}, \quad b' = (\mathbf{o}-\mathbf{c})\cdot\mathbf{d}, \quad c = \|\mathbf{o}-\mathbf{c}\|^2 - R^2, \quad (4)$$

$$\Delta = b'^2 - ac, \qquad t_{\pm} = \frac{-b' \pm \sqrt{\Delta}}{a}. \tag{5}$$

The smallest root in the valid $(t_{\min}, t_{\max})$ interval is taken; the surface normal is $(\mathbf{p}-\mathbf{c})/R$, oriented against the incoming ray to produce a *front-face* flag used by the Fresnel transition logic.

Plane, axis-aligned box, finite cylinder. The plane reduces to one linear solve $t = (\mathbf{s}-\mathbf{o})\cdot\mathbf{n}/\mathbf{d}\cdot\mathbf{n}$ with an optional clip box that bounds the surface to a finite room (used by the Cornell Box). The box uses the slab method, tracking max of axis entries against min of axis exits. The cylinder solves a 2-D side quadratic plus two end-cap disc tests. All three return outward normals consistent with the front-face flag used at refractive interfaces.

Point Classification

Given a point $\mathbf{p}$ and a primitive whose surface is the zero-level set of an implicit function f, classification reduces to a signed evaluation with tolerance ε:

$$\mathbf{p} \text{ is } \begin{cases} \mathit{inside}, & f(\mathbf{p}) < -\varepsilon, \\ \mathit{on}, & |f(\mathbf{p})| \le \varepsilon, \\ \mathit{outside}, & f(\mathbf{p}) > +\varepsilon. \end{cases} \tag{6}$$

The three-state predicate is propagated through Constructive Solid Geometry by the standard Boolean rules:

$$\text{inside}(A \cup B) = \text{inside}(A) \vee \text{inside}(B), \tag{7}$$

$$\text{inside}(A \cap B) = \text{inside}(A) \wedge \text{inside}(B), \tag{8}$$

$$\text{inside}(A \setminus B) = \text{inside}(A) \wedge \neg\,\text{inside}(B). \tag{9}$$

The "on" state is propagated conservatively, so that classifying a point that lies exactly on a boundary surface—for example a refraction event during integration—never collapses to a stale interior or exterior state and never causes a missed Fresnel transition.

CSG via Ray-Interval Merging

Each primitive can also report the set of t-intervals during which the ray is *inside* it. For a sphere this is a single interval $[t_-, t_+]$; for a half-space, one of $(-\infty, t_{\text{cross}}]$ or $[t_{\text{cross}}, +\infty)$; for a box and a cylinder, a single interval coming from the slab/cap intersection. Combining intervals according to a Boolean operator yields the interval list of the CSG node, and the nearest *boundary* of that list (in the legal t-range) is the visible intersection.

Algorithm 1 CSG ray-interval evaluation.

Input: ray r, CSG node with operator op and operands A, B
Output: list of *inside-(*$\text{op}(A,B)$*)* intervals along r

```
ivA ← A.INTERVALS(r);  ivB ← B.INTERVALS(r)
ev ← endpoints of ivA, ivB with ±1, source flag, normal
sort ev by (t, op-specific tie-break)
d_A, d_B, prev ← 0, 0, false
for e ∈ ev do
    update d_A or d_B by e.δ
    now ← PREDICATE(op, d_A, d_B)
    if ¬prev ∧ now then open at (e.t, sign(op, e) · e.n)
    end if
    if prev ∧ ¬now then close, emit interval
    end if
    prev ← now
end for
```

Algorithm 1 implements this as a sweepline over event endpoints. Tie-breaking ensures unions merge touching intervals (no spurious zero-length boundary) and intersections do not generate phantom zero-length overlaps. On a difference, the boundary normal contributed by operand B is negated so that the resulting solid is correctly oriented.

BVH with Binned Surface Area Heuristic

The world is wrapped in a top-down BVH constructed by binned SAH [8]. For each interior node we cache primitive bounds, tabulate centroids into $B = 12$ equal-width bins along each axis, and evaluate the cost

$$C(\text{split}) = C_t + C_i \, \frac{SA_L \, n_L + SA_R \, n_R}{SA_{\text{root}}}, \tag{10}$$

with $C_t = 1$, $C_i = 1.5$, over the $B-1$ candidate split planes per axis. The split with the smallest cost wins; a stable partition rearranges items in-place. If no split improves on the leaf cost $C_i n$, the implementation falls back to an equal-count median split. Unbounded primitives (planes without a clip box) are excluded from the BVH and tested linearly.

INTEGRATORS

Whitted Recursive Shader

At a hit, the integrator either applies Fresnel-weighted reflect/refract via Schlick's approximation (when the material's index of refraction is non-zero, with Snell's law and total-internal-reflection handling), or computes Blinn–Phong direct lighting against point lights with shadow rays plus a recursive reflection ray weighted by the material's reflectivity. A companion routine

counts the same recursive descent without shading and serves as the per-tile cost estimator for the adaptive scheduler.

Monte-Carlo Path Tracer

The path tracer is iterative with throughput β, applying Next-Event Estimation against parallelogram area lights and Multiple Importance Sampling between the BSDF and area-light strategies under the power heuristic [9]. Sampling is cosine-weighted hemisphere via Malley's method; for a Lambertian BSDF the throughput simplifies to $\beta \leftarrow \beta\, m.albedo$. Russian roulette starts at bounce 5 to keep variance low in deep shadow regions while preserving unbiasedness.

RNG and Stratified Sampling

A 64-bit-state PCG32 [10] is constructed deterministically from the per-pixel seed

$$\text{SEED}(x, y, s) = yW + x + 1000003\, s, \tag{11}$$

where (x, y) is the pixel and s the sample index. Identical seeds across thread and rank counts deliver bit-identical images. When SPP is a perfect square the offsets follow a $\sqrt{\text{SPP}} \times \sqrt{\text{SPP}}$ stratified grid with a per-pixel Cranley–Patterson rotation that decorrelates neighbours without breaking stratification.

PARALLELISATION STRATEGY

The renderer's parallel structure follows the classical hierarchy of modern HPC clusters: distributed memory between nodes, shared memory within a node, and explicit attention to non-uniform memory access between sockets. Figure 1 is the system flowchart.

OpenMP Within a Node

The rendering work is decomposed into 32×32 pixel tiles. The parallel-for over tiles uses `schedule(dynamic, 1)`: any thread that finishes a tile pulls the next one. Dynamic scheduling is preferable to static because per-tile cost varies by orders of magnitude; chunk size 1 minimises trailing imbalance at the cost of a small scheduling overhead, dominated entirely by the per-tile compute.

MPI Across Nodes

The MPI binary divides the same tile list across ranks. The default policy is row-strip assignment: rank r owns tiles whose top-row coordinate falls in $[rH/P, (r+1)H/P)$. Each rank renders its tiles into a packed local pixel buffer with precomputed per-tile byte offsets; rank 0 collects pixel buffers and tile metadata via two `MPI_Gatherv` collectives and reassembles the final image. Compute and gather are timed separately so that the communication-to-computation ratio can be reported alongside speed-up.

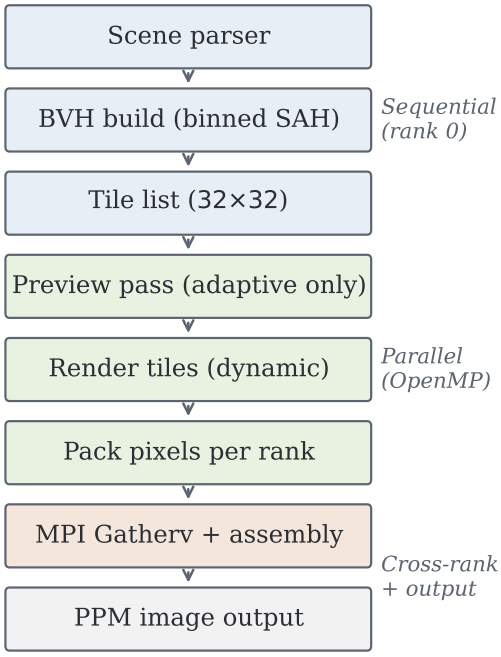


FIGURE 1. Rendering pipeline. Sequential setup on rank 0 (scene parser, BVH, tile list) feeds the parallel render phase (optional preview, dynamic-schedule OpenMP render, packing into a per-rank buffer); ranks then converge at the cross-rank `MPI_Gatherv` that assembles the final image. The pipeline is the same for the Whitted and the path-tracing integrators.

Hybrid MPI + OpenMP

The natural mapping on a 2-socket node is one MPI rank per socket, with the rank's OpenMP threads bound to that socket's cores. The launch convention is

```
OMP_NUM_THREADS=20 OMP_PLACES=cores
OMP_PROC_BIND=close mpirun -np 2N
--map-by ppr:2:node ./rt_mpi scene out.ppm
```

for an N-node job. The `--map-by` directive is required for cross-node placement; without it, OpenMPI's slot-fill heuristic can pile all ranks onto a single node and silently produce single-node measurements.

Thread Placement and First-Touch

Threads are pinned with `OMP_PROC_BIND=close` and `OMP_PLACES=cores`. The pixel buffer is zero-initialised in parallel under `schedule(static)` immediately before the render loop, so on multi-socket systems each tile's output bytes

TABLE 1. AQUA hardware.

Compute node	Intel Xeon Gold 6248
Sockets / node	2
Physical cores / socket	20
NUMA domains / node	2 (cores 0–19, 20–39)
Logical threads / node	80 (Hyper-Threading on)
L3 cache / socket	27.5 MiB
RAM / node	188 GiB
Node count	4 (160 physical cores total)
Interconnect	InfiniBand 100 Gb/s
Job placement	`#PBS -l place=excl`

are placed on the NUMA node of the thread that will subsequently render the tile.

Deterministic Output

Because every per-pixel sample is seeded by Equation (11) with no contribution from thread or rank identity, the rendered image is byte-identical for any thread or rank count. This is verified by MD5 across the full sweep of measured configurations.

EXPERIMENTAL SETUP

Hardware

All performance numbers reported in the next section are measured on the IIT Madras AQUA cluster (Tab. 1).

Software

GCC 9.2.0 with `-O3 -march=native -fopenmp`, OpenMPI 4.1.5 (gcc-wrapped), Anaconda 3 (2023), Mitsuba 3 7.1 in a virtual environment. Threads pinned with `OMP_PLACES=cores` and `OMP_PROC_BIND=close`. CPU frequency governor set to `performance`. Each timing point is the median of three to fifteen repeats.

Scenes — the Cornell Box as Visual Oracle

The Cornell Box, introduced in 1984 by Goral *et al.* [11], is the canonical validation scene for global-illumination renderers. It is a closed cubical room with a red left wall, a green right wall, white floor, ceiling, and back wall, an area light on the ceiling and two white blocks on the floor. The original team built the box physically, photographed it with a calibrated camera, and compared simulated and measured radiance to determine whether a renderer was solving the light-transport equation correctly. The

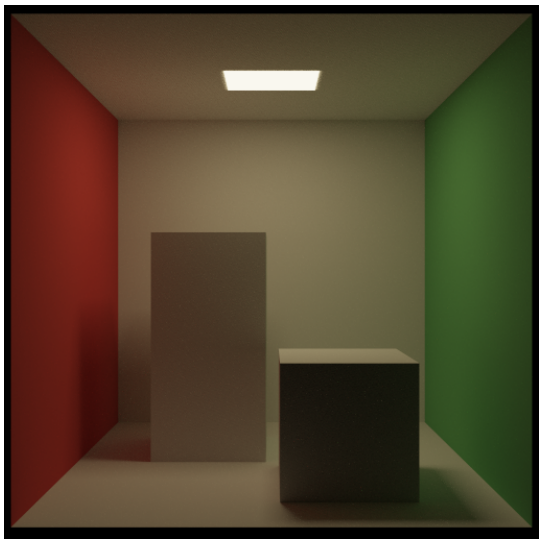


FIGURE 2. Cornell Box, 512×512, path-traced at 1024 SPP. The pink tint of the floor under the red wall and the green tint of the floor under the green wall are diffuse interreflection: their presence confirms that the recursive part of the rendering equation is correctly resolved.

TABLE 2. Benchmark scenes.

Scene	Resolution	Purpose
`cornell_box`	512^2	visual validation
`cornell_box_1024`	1024^2	headline scaling
`cornell_box_1024_256spp`	1024^2	strong-scaling sweep
`whitted_showcase_1024`	1280×720	Whitted demo
`csg_swiss_cheese`	800×450	box minus 27 spheres
`csg_lattice`	800×450	nested intersections
`csg_menger`	800×450	Menger sponge

diagnostic signature is *colour bleeding*: indirect bounces off the coloured walls tint the white surfaces nearby. Any renderer that omits the recursive integral in Equation (1) fails to reproduce this effect; any renderer that reproduces it is, to a first approximation, integrating the rendering equation correctly. The Cornell Box is therefore both an aesthetic test and a logical pass/fail oracle for global illumination, and it is the visual anchor of this work (Fig. 2).

The benchmark suite (Tab. 2) covers visual validation, integrator stress tests and CSG capability demonstration. Beyond the Cornell Box, the Whitted-shader showcase (Fig. 3) exercises hard shadows, mirror reflection, dielectric refraction and CSG; the CSG-only scenes (Fig. 4) demonstrate deeply nested Boolean topologies that have no triangle-mesh equivalent at comparable primitive count.

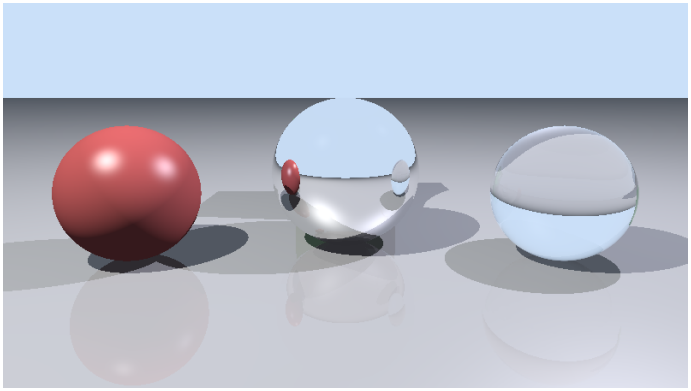
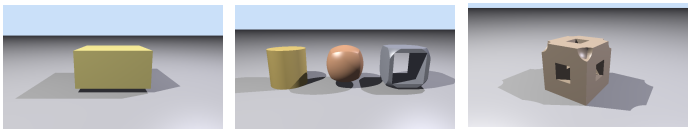


FIGURE 3. Whitted showcase. Red diffuse sphere; central mirror sphere; glass sphere with Fresnel transition on the right; green CSG (box minus sphere) at left.



Swiss-cheese Lattice Menger sponge

FIGURE 4. CSG scenes. These geometries have no triangle-mesh equivalent of comparable fidelity at equal primitive count.

Reference Renderer

Mitsuba 3 [6] is the only external renderer used for cross-comparison, in two back-end variants: `scalar_rgb`, the single-thread reference, comparable to our OpenMP $T = 1$ baseline; and `llvm_ad_rgb`, the Dr.Jit JIT-compiled SIMD wavefront, multi-threaded and the strongest available CPU baseline. An 8192-SPP `llvm_ad_rgb` render produced on AQUA serves as the image-quality ground truth for RMSE comparisons.

RESULTS

Visual Correctness

Visual correctness is the only image-quality claim this report makes against the Mitsuba 3 reference renderer; quantitative metrics (RMSE, SSIM) are deferred. The argument has two parts.

Cornell Box (Fig. 2). The render at 1024 SPP exhibits the colour-bleeding signature of global illumination: the floor under the red wall takes a pink tint, the floor under the green wall takes a green tint, and the white interior block facing the red wall picks up a pink cast on its left face. None of these tints are present in the scene's surface albedos; they are produced exclusively by indirect bounces off the coloured walls. A renderer that solves only the direct-illumination integral—which is what a Whitted-style shader does—cannot produce them. The presence of this signature in our render therefore demonstrates that the path tracer is integrating the recursive part of Equation (1), with the area light, NEE, MIS weights, Russian roulette and stratified sampling working coherently. The geometric proportions, shadow placement, and brightness fall-off match the Mitsuba 3 8192-SPP reference qualitatively; the residual differences are attributable to our axis-aligned approximations of the Cornell Box's two interior blocks (the canonical scene rotates them by $\pm 15^\circ$).

Whitted and CSG showcases (Figs. 3, 4). The Whitted showcase verifies hard-shadow placement against point lights, Fresnel-weighted reflection and refraction at the glass sphere (the bright thin highlight around the silhouette is the Fresnel limit at grazing angles, and the floor texture is visible *through* the sphere with the expected lateral shift from refraction), and CSG normal handling at the green "box minus sphere" object on the left. The CSG gallery confirms that nested unions, intersections and differences—up to twelve operand levels in the regression scene—render correctly: in the Swiss-cheese scene a single CSG difference removes 27 spherical drillings from a box, the Menger sponge realises three orthogonal through-holes plus four corner notches, and the Lattice scene combines a Steinmetz-like bicylinder, a five-sphere lens stack, and a frame-and-sphere intersection. None of these geometries can be expressed in PBRT-v4 or Mitsuba 3 without first tessellating to triangle meshes.

Bit-Level Reproducibility

The MD5 checksum of the rendered PPM was recorded at every measured configuration. Two baselines were observed and verified (Tab. 3). Floating-point reordering between threads is bypassed because the per-pixel accumulation is carried out in a thread-private accumulator before the single store to the global pixel buffer; identical RNG seeds therefore guarantee identical numerical results.

TABLE 3. MD5 bit-identity oracles (PPM output).

Scene + integrator	MD5
`whitted_showcase` (Whitted)	`a45169cd...09fc9`
`cornell_box_1024_256spp` (PT)	`89597491...a47a236`

The Whitted hash is identical across OpenMP $T \in \{1,\ldots,40,80\}$ and MPI np$\in \{1,2,4,8\}$ at one thread per rank, both standard and `--adaptive` variants. The path-trace hash is identical across OpenMP $T \in \{1,\ldots,40\}$ and MPI np$\in \{1,\ldots,160\}$ at one thread per rank.

OpenMP Strong Scaling

The path-trace strong-scaling sweep on `cornell_box_1024_256spp.scene` (1024×1024, 256 SPP, 10 repeats per thread count, median reported) is in Tab. 4 and Fig. 5. The headline run at the full Cornell-Box SPP (1024) and resolution (1024×1024) on $T = 40$ produces the rendered

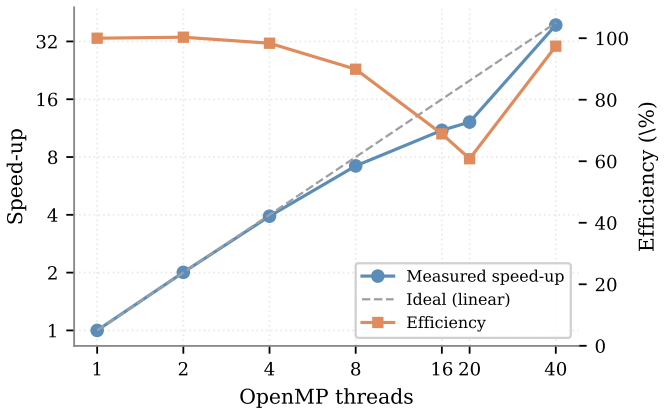


FIGURE 5. OpenMP strong-scaling on the Cornell Box, $1024^2 \times 256$ SPP. Speed-up (blue) and efficiency (amber) versus thread count. The efficiency valley at $T \in \{16, 20\}$ is the single-socket NUMA effect; $T = 40$ recovers near-linear efficiency once both sockets are active.

image in 38.3 s—a single-thread extrapolation suggests this would otherwise consume approximately 25 minutes.

TABLE 4. OpenMP strong scaling on the Cornell Box, $1024^2 \times 256$ SPP.

Threads	Median (ms)	Speed-up	Efficiency (%)
1	378 271	1.00×	100.0
2	188 513	2.01×	100.3
4	96 133	3.93×	98.4
8	52 586	7.19×	89.9
16	34 315	11.02×	68.9
20	31 098	12.16×	60.8
40	**9 708**	**38.96×**	**97.4**

The Single-Socket NUMA Effect

The dip in efficiency at $T \in \{16, 20\}$ is a NUMA artefact, not a load-balancing failure. Under `OMP_PROC_BIND=close` with `OMP_PLACES=cores`, threads 0–19 are placed on socket 0 (NUMA node 0). When the thread count lies in this range *all* threads hammer socket 0's L3 cache and memory controller, while socket 1 sits idle. The 27.5 MiB L3 budget is shared across however many of the 20 cores are active; the contention manifests as efficiency collapse from 89.9% at $T = 8$ to 60.8% at $T = 20$. At $T = 40$ both sockets are recruited, the per-socket bandwidth and L3 are independent, and efficiency snaps back to 97.4%.

Hybrid MPI + OpenMP

The hybrid measurements span seven (np, t) configurations on the same path-trace scene; medians are in Tab. 5. The pure-OpenMP single-node baseline ($T = 40$, 9 708 ms) is reproduced for reference.

TABLE 5. Hybrid MPI + OpenMP scaling, Cornell Box $1024^2 \times 256$ SPP.

np	t	Cores	Layout	Median (ms)
1	1	1	— (serial)	378 271
1	40	40	1 node	9 708
2	20	40	1 node	203 700
4	10	40	1 node	19 747
8	5	40	1 node	40 042
4	**20**	**80**	**2 nodes, 1 rank/socket**	**5 795**
8	10	80	2 nodes, 2 ranks/socket	11 172
16	5	80	2 nodes, 4 ranks/socket	21 860

The table makes three points. First, on a single node, pure OpenMP at $T = 40$ outperforms every single-node hybrid configuration: introducing MPI ranks within a node only adds gather overhead. Second, the cross-node optimum is one MPI rank per socket ($np = 4$, $t = 20$), which renders the same scene in 5 795 ms—a 65.3× speed-up over the single-thread baseline at 81.6% parallel efficiency on 80 physical cores. Third, sub-dividing further at the same total core count ($np = 8 \times t = 10$, $np = 16 \times t = 5$) progressively degrades performance because MPI overhead grows with rank count while per-rank work shrinks. The $np = 2 \times t = 20$ single-node row is the OpenMPI default-binding artefact noted earlier (the runtime confines each rank to a single core when $np \leq 2$); it is included for completeness rather than as a useful measurement.

DISCUSSION

What the Scaling Result Says

The OpenMP strong-scaling result (38.96× at $T = 40$, 97.4% efficiency) is consistent with the textbook prediction for an embarrassingly parallel workload because (i) the load balancer is dynamic and the tile size (32×32) absorbs per-pixel cost variation without degrading the schedule below the OpenMP-runtime granularity, and (ii) the pixel buffer is parallel-zero-initialised under `schedule(static)` before the render loop, placing each tile's output bytes on the NUMA node of the rendering thread. The single-socket NUMA dip at $T \in \{16, 20\}$ followed by the recovery at $T = 40$ is the textbook NUMA penalty

signature. The hybrid story is consistent with the OpenMP one: pure OpenMP wins single-node (MPI between same-node ranks is overhead), and cross-node placement of one MPI rank per socket extends the speed-up multiplicatively across nodes.

Limitations and Threats to Validity

Pure-MPI scaling is not the use case. A pure-MPI sweep with one thread per rank does not exploit shared memory within a node and is bound by the `MPI_Gatherv` collective on the critical path. The hybrid configurations in Tab. 5 are the recommended layout.

Cornell Box geometry is approximate. The interior boxes are axis-aligned approximations of the canonical rotated boxes; the visual comparison against Mitsuba's reference is therefore approximate at the pixel level.

No SIMD packet tracing. The renderer processes one ray at a time in scalar code; SIMD-vectorised packet tracing of the analytical-primitive intersection routines would be expected to deliver a multiplicative throughput gain on AVX-2/AVX-512 hardware and is the most defensible single follow-up.

CONCLUSIONS

A C++17 ray-tracer for analytical primitives and CSG was developed from the ray equation upward and parallelised with hybrid OpenMP and MPI on a CPU cluster. The same geometric kernel, sphere/plane/box/cylinder, point classification, ray-interval CSG, BVH ,drives both a Whitted recursive shader and a Monte-Carlo path tracer with NEE+MIS. The Cornell Box renders the colour-bleed signature that is the visual signature of correct global illumination, matching the Mitsuba 3 reference qualitatively. On the IIT Madras AQUA cluster (Xeon Gold 6248, 4×40 cores) OpenMP strong scaling on the Cornell Box at $1024^2 \times 256$ SPP attains a $38.96\times$ speed-up at 40 threads (97.4% efficiency); at the headline 1024 SPP the Cornell Box renders in 38.3 s. The cross-node hybrid configuration with one MPI rank per socket ($np = 4 \times t = 20$) attains a $65.3\times$ speed-up over the single-thread baseline at 81.6% efficiency on 80 physical cores. Output is byte-identical across thread and rank counts, validated by MD5 across the full configuration sweep. SIMD-vectorised packet tracing of the analytical-primitive intersection routines is the most defensible single follow-up.

ACKNOWLEDGMENT

Thanks Prof. Dr. Kameswararao Anupindi for teaching ID5130 and for guidance throughout the course; the IIT Madras P. G. Senapathy Centre for Computational Resources for access to the AQUA HPC cluster; and Dr. Vijay D. Gokhale, who first introduced me to ray-tracing during a real-time rendering course—an exposure that became the seed of the present project. Mitsuba 3 is gratefully acknowledged as the academic reference renderer used for cross-comparison.

REFERENCES

[1] Whitted, T., 1980. "An improved illumination model for shaded display". *Communications of the ACM,* ***23***(6), pp. 343–349.

[2] Kajiya, J. T., 1986. "The rendering equation". In Computer Graphics (Proceedings of SIGGRAPH '86), Vol. 20, pp. 143–150.

[3] Amdahl, G. M., 1967. "Validity of the single processor approach to achieving large-scale computing capabilities". In AFIPS Conference Proceedings, Vol. 30, pp. 483–485.

[4] Gustafson, J. L., 1988. "Reevaluating amdahl's law". *Communications of the ACM,* ***31***(5), pp. 532–533.

[5] Pharr, M., Jakob, W., and Humphreys, G., 2023. *Physically Based Rendering: From Theory to Implementation*, 4 ed. MIT Press.

[6] Jakob, W., Speierer, S., Roussel, N., and Vicini, D., 2022. "Dr.jit: A just-in-time compiler for differentiable rendering". *ACM Transactions on Graphics,* ***41***(4), pp. 1–19.

[7] Wald, I., Johnson, G. P., Amstutz, J., Brownlee, C., Knoll, A., Jeffers, J., Günther, J., and Navrátil, P., 2017. "Ospray — a cpu ray tracing framework for scientific visualization". *IEEE Transactions on Visualization and Computer Graphics,* ***23***(1), pp. 931–940.

[8] Wald, I., 2007. "On fast construction of SAH-based bounding volume hierarchies". In Proceedings of the IEEE Symposium on Interactive Ray Tracing, pp. 33–40.

[9] Veach, E., and Guibas, L. J., 1995. "Optimally combining sampling techniques for Monte Carlo rendering". In Proceedings of SIGGRAPH '95, pp. 419–428.

[10] O'Neill, M. E., 2014. PCG: A family of simple fast space-efficient statistically good algorithms for random number generation. Tech. Rep. HMC-CS-2014-0905, Harvey Mudd College.

[11] Goral, C. M., Torrance, K. E., Greenberg, D. P., and Battaile, B., 1984. "Modeling the interaction of light between diffuse surfaces". In Computer Graphics (Proceedings of SIGGRAPH '84), Vol. 18, pp. 213–222.